IGWT LECTURES

Computational Geometry

Interactive Graphics and Web Technologies

Combined notes from this folder. Generated from Markdown source.

Computational Geometry — Lecture notes

Parent plan: 04 Computational Geometry

Teach these in order. Each note is one 75-minute lecture plus the live-coding, lab, homework, and quiz for that week.

Week	Note
1	Week 01 What Computational Geometry Is
2	Week 02 Geometric Primitives
3	Week 03 Convexity and Polygons
4	Week 04 Convex Hull I
5	Week 05 Convex Hull II
6	Week 06 Sweep Line Intersection
7	Week 07 Polygon Triangulation
8	Week 08 Midterm and DCEL
9	Week 09 Point Location and Range Search
10	Week 10 Voronoi Diagrams
11	Week 11 Delaunay Triangulation
12	Week 12 Closest Pair and Survey
13	Week 13 Graphics Applications
14	Week 14 Project Studio
15	Week 15 Presentations

Week 1 — What computational geometry is

Time: 75 min lecture + 60 min live coding
Kernel this week: `orient(a, b, c)`
Board first: one picture of three points and a signed area

Timing

Minutes	Do this
0–10	Why this course exists (graphics already is geometry)
10–25	Four flagship problems
25–40	Predicates vs constructions
40–55	Degeneracy and floating point
55–70	Course contract and 15-week map
70–75	Preview `orient`, then stand up for live coding

Learning goals

By the end of the lecture a student can:

1. Name the five problem families of this course.
2. Distinguish a **predicate** from a **construction**.
3. Give three degenerate cases that break naive code.
4. Compute the 2D cross product and interpret its sign.
5. Explain why `atan2` is the wrong primitive for “left of a line.”

---

1. Opening (10 min)

Computer graphics is full of questions that look continuous but must be answered with discrete algorithms:

- Does this click hit the polygon?
- Do these two walls cross?
- What is the outline of this point cloud?
- Which triangle contains the ray?
- Who is the nearest site to this pixel?

Computational geometry is the study of **algorithms for geometric objects**: points, segments, polygons, and later meshes.

This course is 2D-first. Almost every 3D graphics test (ray–triangle, frustum, collision) is the same idea with one extra coordinate.

Write on the board:

```
input: finite set of points / segments / polygons
output: a combinatorial structure + a few constructed points
```

We care about:

- **correctness** on ugly inputs
- **complexity** in n
- **visualization** so students can see the invariant

---

2. Four flagship problems (15 min)

Draw each one. Do not implement yet.

Closest pair

Input: n points.
Output: the two that are nearest.

Naive: try all pairs, $\Theta(n^2)$.
This course: divide and conquer, $O(n \log n)$ (Week 12).

Convex hull

Input: n points.
Output: the smallest convex polygon containing them.

Mental image: stretch a rubber band around nails.

Segment intersection

Input: n line segments.
Output: all crossing points, or yes/no.

Naive: every pair, $\Theta(n^2)$.
This course: sweep line (Week 6).

Point in polygon

Input: a polygon P and a query point q.
Output: inside / outside / on boundary.

Used every time a user clicks a shape.

Fifth family, named now, taught later: proximity (Voronoi / Delaunay) and search (kd-tree / BVH).

---

3. Predicates vs constructions (15 min)

A **predicate** returns a discrete answer.

Examples:

- Is c left of directed line ab?
- Do segments ab and cd intersect?
- Is triangle abc oriented counterclockwise?

A **construction** returns new geometry.

Examples:

- The intersection point of two lines
- The circumcenter of three points
- The convex hull polygon

Teaching rule: implement predicates first. Constructions inherit their errors.

The fundamental 2D predicate is orientation.

For points a, b, c define

```
cross(b - a, c - a) = (bx - ax)(cy - ay) - (by - ay)(cx - ax)
```

Sign	Meaning
> 0	c is to the **left** of directed line ab (CCW)
= 0	a, b, c are **collinear**
< 0	c is to the **right** of ab (CW)

This value is also twice the signed area of triangle abc.

```
area(abc) = cross(b - a, c - a) / 2
```

Why not angles?

`atan2` is slow, wraps at ±π, and is unstable near the branch cut. The cross product uses four subtractions and two multiplies. It is the primitive of the whole course.

Pseudocode

```
orient(a, b, c):
    v = (b.x − a.x) * (c.y − a.y) − (b.y − a.y) * (c.x − a.x)
    if v > EPS: return LEFT
    if v < −EPS: return RIGHT
    return COLLINEAR
```

This week, mention `EPS` but do not pretend it solves everything. Week 13 returns to robustness.

---

4. Degeneracy and floating point (15 min)

A case is **degenerate** when a predicate that is “usually” nonzero becomes zero, or when objects coincide.

Show these four pictures:

1. Three collinear points on a hull
2. Two segments that touch at an endpoint (T-junction)
3. Two overlapping collinear segments
4. Duplicate vertices in a polygon

Then show the floating-point surprise:

```
a = (0, 0)
b = (1, 0)
c = (1e20, 1)
```

The true orientation is LEFT, but `1e20` can swallow the `1` in IEEE-754 double. Naive code reports COLLINEAR.

Course policy on degeneracy:

- Detect it. Do not hope it will not happen.
- Define a policy (keep collinear hull points or drop them).
- Write a test for it.
- Never use `== 0` on a computed float without saying you are using an epsilon.

Exact arithmetic and Shewchuk predicates are named now, required later only as reading.

---

5. Course contract (15 min)

What we will implement

Weeks 1–7: kernel, polygons, hulls, sweep, triangulation.
Week 8: midterm + DCEL.

Weeks 9–12: search, Voronoi, Delaunay, closest pair.
Weeks 13–15: graphics applications and the project.

What we will not implement

Full Fortune, Kirkpatrick point location, 3D Delaunay, CGAL kernels. We will **name** them.

How a week works

Lecture → live coding on a canvas → lab → homework → 10-minute quiz.

Assessment reminder

Labs 25%, homework 20%, quizzes 10%, midterm 15%, project 30%.

Language

JavaScript + Canvas, unless the department has already standardized on Python. The math does not change.

Live coding (60 min)

Build the shared visualizer from zero.

Must work by the end of class:

1. Click adds a point.
2. Drag moves the nearest point.
3. Reset clears.
4. Coordinates drawn next to each point.
5. If the student (or you) has selected three points A, B, C:
   - draw segment AB
   - fill triangle ABC with green / red / gray by `orient`
   - print `LEFT`, `RIGHT`, or `COLLINEAR` and the raw cross value

Talk while typing:

- “I subtract A first so the test is translation-invariant.”
- “I do not call `Math.atan2`.”
- “I print the raw value so we can see near-zero cases.”

Starter they leave with:

```
export function cross(ax, ay, bx, by, cx, cy) {
  return (bx - ax) * (cy - ay) - (by - ay) * (cx - ax);
}

export function orient(a, b, c, eps = 1e-9) {
  const v = cross(a.x, a.y, b.x, b.y, c.x, c.y);
  if (v > eps) return 1;
  if (v < -eps) return -1;
  return 0;
}
```

Lab (2–3 hours)

Using the visualizer:

1. Implement `distance(a, b)` and `midpoint(a, b)`.
2. Display the signed area of ABC.
3. Add a button: "make C collinear with AB" (project C onto AB).
4. Write 4 console tests: left, right, collinear-between, collinear-beyond.

Done when a TA can click three points and see sign, area, and distance without opening the console.

Homework

Due start of Week 2.

1. Implement `orient` with 8 unit tests. Required cases:
   - left, right
   - collinear, C between A and B
   - collinear, C beyond B
   - C = A
   - very small triangle
   - a translated copy of a previous case (invariance)
2. Written (half page): why the cross product is better than comparing polar angles for the left-of-line test.
3. Written: give one floating-point input you expect to be fragile. You do not need to break IEEE; you need to explain the risk.

Quiz (10 min, start of Week 2 or end of Week 1)

1. Compute `cross` for A=(0,0), B=(2,0), C=(1,3). Inside/left/right? (2 pts)
2. Is "the intersection point of two lines" a predicate or a construction? (2 pts)

3. Name two degenerate cases for segment intersection. (3 pts)
4. Why is sorting polar angles a bad replacement for `orient`? One sentence. (3 pts)

Common mistakes

- Using angles.
- Testing `cross === 0` and then arguing the code is "exact."
- Forgetting that AB is **directed**. `orient(a,b,c)` is not `orient(b,a,c)`.
- Building a visualizer that cannot move points. Degeneracy is easier to show by dragging.

Board drawings

1. Triangle ABC with an arrow on AB and a + / − on C.
2. Four flagship problem thumbnails.
3. The 15-week map as five boxes: primitives, hulls, sweep/tri, proximity/search, project.

Week 2 — Geometric primitives

Time: 75 min lecture + 60 min live coding
Kernel this week: `onSegment`, `segmentsIntersect`, `pointInPolygon`
Board first: the five objects (point, segment, ray, line, polygon)

Timing

Minutes	Do this
0–10	Quiz from Week 1, then objects
10–25	Cross product recap and `onSegment`
25–45	Segment–segment intersection, all four answers
45–65	Point in polygon: ray casting and winding
65–75	Bounding boxes; list the failure cases

Learning goals

1. Define point, vector, segment, ray, line, and simple polygon.
2. Implement `onSegment` using orientation + bounding box.
3. Classify two segments as `proper`, `touch`, `overlap`, or `none`.
4. Explain ray casting and the vertex-hit bug.
5. Use an AABB as a reject test, not as the intersection test.

---

1. Objects (10 min)

Object	Definition we will use
Point	`(x, y)`
Vector	difference of two points
Segment `ab`	points `(1−t)a + t b` for `t ∈ [0, 1]`
Ray `ab`	same, `t ≥ 0`
Line `ab`	same, `t ∈ ℝ`
Polygon	cyclic sequence of vertices `v0…v_{n−1}` with edges `vi → v_{i+1}`

A polygon is **closed**. Edge `v_{n−1} → v0` always exists.

We do **not** yet require convexity. We do require that students can walk edges in order.

---

2. `onSegment` (15 min)

C lies on segment AB iff:

1. `orient(A, B, C) = 0` (collinear)
2. C is inside the axis-aligned bounding box of AB

```
onSegment(c, a, b):
    if orient(a, b, c) != 0: return false
    return min(a.x, b.x) - EPS <= c.x <= max(a.x, b.x) + EPS
       and min(a.y, b.y) - EPS <= c.y <= max(a.y, b.y) + EPS
```

Why the box? Collinear is not enough. C can sit on the line but outside the segment.

Draw:

```
A ---- B          C          (collinear, not on segment)
A -- C -- B                  (on segment)
A = C ---- B                 (on segment: endpoint)
```

---

3. Segment intersection (20 min)

Let S1 = AB, S2 = CD.

Proper intersection

The segments cross at a point that is **interior** to both.

Predicate (no division):

```
orient(A,B,C) and orient(A,B,D) have opposite signs
AND
orient(C,D,A) and orient(C,D,B) have opposite signs
```

Opposite signs means one LEFT and one RIGHT. Zero is not opposite.

Improper / touch

They share an endpoint, or one endpoint lies in the interior of the other (T-junction).

Use `onSegment`.

Overlap

Collinear and the 1D intervals overlap.

None

Everything else, including “bounding boxes overlap but segments do not.”

Return type for the course

```
{ type: "proper" | "touch" | "overlap" | "none",
  point?: {x, y} }   // required for proper and touch
```

Construction of the proper intersection point

Parametric:

```
A + t (B − A) = C + u (D − C)
```

Solve the 2×2 system. The denominator is `cross(B−A, D−C)`. If it is near zero, they are parallel; do not divide.

Teach this order: classify with predicates, construct the point only if `proper` or `touch`.

Four pictures (draw all four)

1. Crossing X — `proper`
2. T-junction — `touch`
3. Two collinear overlapping intervals — `overlap`
4. Two disjoint segments whose AABBs overlap — `none`

---

4. Point in polygon (20 min)

Ray casting (even–odd)

Cast a ray from q to +∞ in x (or a slightly tilted ray). Count crossings with polygon edges. Odd ⇒ inside.

The vertex bug: if the ray hits a vertex, two edges can both count, or neither, and the parity is wrong.

Standard fix (half-open edges): an edge `vi → vj` is counted only if one endpoint is strictly above the ray and the other is on or below (or the symmetric rule). Pick one rule and keep it.

```
pointInPolygonEvenOdd(q, P):
    inside = false
    for each edge (a, b) of P:
        if onSegment(q, a, b): return BOUNDARY
        if (a.y > q.y) != (b.y > q.y):
            xHit = a.x + (q.y − a.y) * (b.x − a.x) / (b.y − a.y)
            if q.x < xHit: inside = !inside
    return inside ? INSIDE : OUTSIDE
```

The condition `(a.y > q.y) != (b.y > q.y)` is the half-open trick.

Winding number

Walk the boundary. Add +1 for CCW windings around q, −1 for CW. Nonzero ⇒ inside.

Winding handles holes and overlapping windings more cleanly. Even–odd is enough for simple polygons in this course.

Boundary

Always test `onSegment` first. “Inside” vs “on” matters for picking and for clipping.

---

5. Bounding boxes (10 min)

The **AABB** of a segment or polygon is

```
(min x, min y, max x, max y)
```

If two AABBs are disjoint, the objects cannot intersect. The converse is false. Picture 4 from Section 3 is the counterexample.

In graphics this is the broad phase. Computational geometry still needs the narrow-phase predicate.

Live coding (60 min)

Implement `segmentsIntersect` in the visualizer.

Interaction:

- Two segments, four draggable endpoints
- Color: green proper, orange touch, purple overlap, gray none
- Draw the intersection point when it exists
- Overlay both AABBs as dashed rectangles

Script the four cases by dragging live. Pause on the AABB-overlap / no-intersection case and say: “this is why the box is not the algorithm.”

Leave `pointInPolygon` as the lab. Show one failing ray-through-vertex if time remains.

Lab

1. Implement `pointInPolygon` (even–odd) on a concave polygon the user can draw.
2. Required test shapes:
   - convex

   - concave C-shape
   - query on a vertex
   - query on an edge
   - query whose ray hits a vertex
3. Display `INSIDE` / `OUTSIDE` / `BOUNDARY`.

Done when the TA can put a point on a vertex and get `BOUNDARY`, not a flicker between in and out.

Homework

1. Implement `segmentsIntersect` returning the four-way type. Tests for all four pictures plus:
   - shared endpoint
   - identical segments
   - zero-length segment (A = B)
2. Written: why we classify with orientations before dividing for the intersection point.
3. Written: give the half-open edge rule in one paragraph.

Quiz (10 min)

1. A=(0,0), B=(4,0), C=(2,2), D=(2,−2). Type of intersection? (2 pts)
2. Why is collinear not enough for `onSegment`? (2 pts)
3. Draw a concave polygon and a point whose naive ray hits a vertex. (3 pts)
4. Two AABBs overlap. Must the segments intersect? Yes/no and one sentence. (3 pts)

Common mistakes

- `== 0` on the cross product.
- Dividing before checking parallel.
- Counting both edges when the ray hits a vertex.
- Treating `touch` as `none` (breaks later DCEL and map overlay).
- Using the AABB as the answer.

Board drawings

1. The four intersection types.
2. Ray casting with a vertex hit, then the same figure with the half-open rule marked.
3. AABB of two disjoint crossing-looking segments that do not meet.

Week 3 — Convexity and polygons

Time: 75 min lecture + 60 min live coding
Kernel this week: polygon classifier + shoelace area
Board first: convex set vs convex polygon vs simple polygon

Timing

Minutes	Do this
0–10	Quiz Week 2
10–25	Convex sets and convex combinations
25–45	Polygon taxonomy
45–60	Same-turn test and supporting lines
60–75	Shoelace formula; why algorithms assume simple

Learning goals

1. Define a convex set and a convex combination.
2. Classify a polygon as convex, simple concave, or self-intersecting.
3. Use the all-turns-same-orientation test.
4. Compute signed area with the shoelace formula.
5. Explain why most algorithms this semester assume a **simple** polygon.

---

1. Convex sets (15 min)

A set $S \subseteq \mathbb{R}^2$ is **convex** if for every pair of points p, q in S, the whole segment pq lies in S.

A **convex combination** of points p1…pk is

```
λ1 p1 + … + λk pk
  where λi ≥ 0 and λ1 + … + λk = 1
```

The **convex hull** of a set is the set of all convex combinations of its points. That sentence is the definition we will use again in Week 4.

Draw:

- a disk (convex)
- a banana (not convex: the chord leaves the set)

- a triangle (convex)
- a star polygon (not convex)

Supporting line: a line L through the boundary of S such that S lies entirely in one closed half-plane of L. Convex sets have a supporting line at every boundary point. This is the geometric engine of gift wrapping.

---

2. Polygon taxonomy (20 min)

A polygon is a closed chain of $n \geq 3$ vertices.

Name	Meaning
Simple	edges meet only at shared endpoints; no self-crossings
Convex	simple, and the interior is a convex set
Concave (non-convex)	simple, but at least one reflex vertex
Complex / self-intersecting	edges cross (bowtie, star)
Weakly simple	touches itself but does not cross (mention only)

A **reflex vertex** has interior angle > 180°. Equivalently, the turn at that vertex has the **opposite** orientation from the polygon's overall orientation.

How to get the overall orientation

Signed area (Section 4). Positive ⇒ CCW in our convention. Then a vertex i is reflex iff

```
orient(v_{i−1}, v_i, v_{i+1})
```

disagrees with that sign.

Pictures (draw all four)

1. Convex hexagon
2. Concave "C"
3. Bowtie (two triangles sharing a crossing)
4. Regular star {5/2}

Ask: which of our later algorithms may run on each?
Answer: hull of the **vertices** always exists. Ear clipping and even–odd filling assume simple. Self-intersecting polygons must be rejected or repaired.

---

3. The same-turn test (15 min)

Theorem (teaching statement).
A simple polygon is convex if and only if every consecutive triple `v_{i-1}, v_i, v_{i+1}` has the same orientation, and that orientation is nonzero (no 180° flat vertex, or we treat flat as allowed by policy).

Proof sketch (board).

(⇒) If the polygon is convex, each interior angle is ≤ 180°, so each turn is a left turn if the polygon is CCW (or each is a right turn if CW).

(⇐) If every turn has the same sign, the polygon is locally convex. For a **simple** polygon this implies the interior is a convex set: any chord between two vertices that left the interior would force a reflex chain. (Do not pretend this is a full proof without simplicity. The bowtie has consistent turns on each triangle and is not a convex polygon.)

Course policy on collinear vertices:

- `FLAT` is allowed in the convex class if you document it.
- For hulls next week we will drop or keep them explicitly.

Algorithm

```
classify(P):
    if n < 3: return INVALID
    if any non-adjacent edges intersect properly: return SELF_INTERSECTING
    sign = 0
    for i in 0..n-1:
        o = orient(v[i-1], v[i], v[i+1])
        if o == 0: continue   // flat, by policy
        if sign == 0: sign = o
        else if o != sign: return SIMPLE_CONCAVE
    return CONVEX
```

Intersection of non-adjacent edges uses Week 2. Adjacent edges share a vertex and must not be reported as `proper`.

---

4. Shoelace formula (15 min)

For a polygon with vertices v0…v_{n-1}:

```
2A = Σ_{i=0}^{n-1} (v_i.x * v_{i+1}.y − v_{i+1}.x * v_i.y)
```

with `v_n = v_0`.

Then `A` is the **signed** area. `|A|` is the geometric area.

Derivation: sum of signed areas of triangles `(origin, v_i, v_{i+1})`. The origin cancels; any origin works.

Uses:

- orientation of the polygon (sign of A)
- reflex-vertex test
- later: ear clipping needs a consistent interior

If A = 0 the polygon is degenerate (collapsed).

---

5. Why "simple" is a precondition (rest of lecture)

Write a table and leave it up for the rest of the semester.

Algorithm	Needs simple?	Needs convex?
Point in polygon (even–odd)	yes, or define a fill rule	no
Ear clipping	yes	no
Convex hull of vertices	no	output is convex
SAT collision	yes	yes (convex pieces)
DCEL overlay	no crossings, or split them	no

Graphics connection: a UI shape or a font outline that self-intersects will triangulate into garbage. Blender / SVG / glTF all silently assume almost-simple input. This course will **reject** self-intersection in the classifier rather than guess.

Live coding (60 min)

User draws a polygon by clicking, Enter to close.

Show:

- signed area and CW/CCW
- each vertex colored: green convex, orange reflex, gray flat
- label: `CONVEX` / `SIMPLE_CONCAVE` / `SELF_INTERSECTING`
- faint convex hull of the vertices (call a stub; real hull is Week 4–5)

Drag a vertex of a convex hexagon until it becomes reflex. Students should see the label flip.

Then drag two non-adjacent edges across each other and show `SELF_INTERSECTING`.

Lab

1. Implement `classify(polygon)` with the algorithm above.
2. Implement shoelace and display area.
3. Highlight reflex vertices.
4. Required drawings: convex, C-shape, bowtie, one with a flat vertex.

Done when the bowtie is not reported as convex.

Homework

1. Written proof, 1 page: “A simple polygon is convex iff every turn has the same orientation.” Use the (⇒) direction fully. For (⇐) give the idea and state that simplicity is required. Draw the bowtie as a counterexample without simplicity.
2. Code: classifier + 6 fixtures (the four drawings plus n=3 and a duplicate-vertex square).
3. Compute by hand the shoelace area of (0,0), (4,0), (4,3), (0,3).

Quiz (10 min)

1. Define convex set in one sentence. (2 pts)
2. Is a star polygon simple? Convex? (2 pts)
3. A 5-gon has turns +, +, −, +, +. Simple. Classification? (2 pts)
4. Shoelace of (0,0), (2,0), (0,2). Signed area? (2 pts)
5. Why does the same-turn test need simplicity? (2 pts)

Common mistakes

- Calling any non-convex polygon “complex.”
- Using interior angles in degrees instead of `orient`.
- Testing adjacent edges for proper intersection and marking a valid polygon as self-intersecting.
- Absolute area only, then losing CW/CCW.
- Treating a bowtie as two convex polygons without splitting at the crossing.

Board drawings

1. Convex set definition (segment stays inside).
2. Four-polygon taxonomy.

3. One reflex vertex with the two incident edges and a marked interior.
4. Shoelace as sum of signed triangles from the origin.

Week 4 — Convex hull I (intuition and slow algorithms)

Time: 75 min lecture + 60 min live coding
Algorithm this week: Jarvis march (gift wrapping)
Board first: rubber band around nails

Timing

Minutes	Do this
0–10	Quiz Week 3
10–25	Definitions: hull, extreme points, supporting line
25–50	Jarvis march, invariant, complexity
50–65	Incremental construction (idea)
65–75	Lower bound: hull is at least sorting

Learning goals

1. Define the convex hull of a finite point set.
2. Identify extreme points and supporting lines.
3. Run Jarvis march by hand and in code.
4. State the complexity $\Theta(n\ h)$ and when that is good or bad.
5. Explain why $\Omega(n \log n)$ is a lower bound in the algebraic decision-tree sense (teaching level).

---

1. Definitions (15 min)

Let S be a finite set of n points in the plane.

The **convex hull** `CH(S)` is the smallest convex set containing S.
Equivalently: the intersection of all convex sets that contain S.
Equivalently: the set of all convex combinations of points of S.

For a finite set, `CH(S)` is a convex polygon whose vertices are points of S.

A point $p \in S$ is **extreme** if p is a vertex of `CH(S)`. Equivalently: there exists a supporting line through p such that all of S lies on one side.

Interior / boundary / hole points: a point strictly inside the hull is not extreme. Collinear points in the middle of a hull edge are extreme only if we keep them (policy).

Course policy (state now, keep through Week 5):

- Remove duplicate points first.
- On a hull edge, **drop** strictly intermediate collinear points. The hull vertices are the two endpoints of that edge.

---

2. Jarvis march (25 min)

Also called **gift wrapping**.

Idea

Start at a guaranteed hull point: the lowest point (smallest y; if tie, smallest x).
Then repeatedly wrap a supporting line around the set until we return to the start.

Algorithm

```
jarvis(S):
    remove duplicates
    if n ≤ 1: return S
    start = lowest-then-leftmost point
    hull = []
    p = start
    do:
        hull.append(p)
        q = first point in S that is not p
        for r in S:
            if r == p: continue
            o = orient(p, q, r)
            if o < 0:                 // r is right of p→q, so q is not the next hull v
                q = r
            else if o == 0 and farther(p, r, q):
                q = r                 // same direction, take the farthest
        p = q
    while p != start
    return hull
```

`farther(p, r, q)` is `dist(p,r) > dist(p,q)`.

If we **drop** intermediate collinear points, the `o == 0` branch should still pick the farthest, so we jump to the end of the collinear run.

Invariant

After k steps, the polyline `hull[0]…hull[k-1]` is a prefix of the hull boundary, and the current supporting ray has S on its left (if we wrap CCW).

We wrap CCW if "r is more left" updates q. The code above wraps by rejecting right turns, i.e. the next edge is the one with all points to the left or on it.

Complexity

Each of the h hull edges scans up to n points.
Time: Θ(n h).
Space: O(h) besides the input.

Input	h	Time
Points in convex position (circle)	n	$\Theta(n^2)$
Points with 3 extreme (triangle + cloud inside)	3	Θ(n)
Random Gaussian	O(log n) expected	about O(n log n)

Jarvis is **output-sensitive**. Good when h is tiny. Bad when the points are already on a circle.

Trace on the board

Use 8 points, 5 on the hull. Walk one candidate at a time. Students should see q jump whenever a righter (or lefter) point appears.

---

3. Incremental construction (15 min)

Idea only. No required implementation.

Add points one by one.

- Maintain `CH` of the points so far.
- If the new point p is inside, do nothing.
- If p is outside, find the two tangents from p to the current hull and replace the visible chain by p.

Finding tangents on a convex polygon is O(log n) with binary search, or O(n) naively. A naive incremental algorithm is $O(n^2)$.

This is the right mental model for 3D incremental hulls. Mention it; do not code it this week.

---

4. Lower bound (10 min)

Claim. Computing the convex hull in 2D takes Ω(n log n) time in the algebraic decision-tree model.

Reduction (teaching).
To sort x1…xn, map each xi to the point `(xi, xi²)` on the parabola $y = x^2$. The hull (lower or full) visits the points in sorted x-order. If hull were o(n log n), sorting would be too.

So Graham / Andrew (next week) are optimal in the worst case. Jarvis is not, unless h is small.

Live coding (60 min)

Implement Jarvis with **step mode**.

Keys:

- `N` — one candidate comparison
- `E` — finish the current hull edge
- `Space` — run to completion

Draw:

- current p (blue)
- current candidate q (orange)
- point r being tested (black)
- accepted hull edges (thick)
- the full set (dots)

Talk:

- “I always start at lowest-then-leftmost, so I never start inside.”
- “When three are collinear I take the farthest, so I obey the drop-middle policy.”

Time a random n = 2000 run vs a circle of n = 400 so students see Θ(n h).

Lab

1. Implement Jarvis march.
2. Buttons: random cloud, random circle, triangle-plus-cloud.
3. Print n, h, elapsed milliseconds.
4. Measure n = 100, 1_000, 10_000 on cloud and on circle. Table in the README.

Done when the circle case is visibly slower than the cloud for the same n.

Homework

1. Implement Jarvis with the course collinear policy and duplicate removal.
2. Written: an input with h = 3 and an input with h = n. State the resulting Θ.
3. Written: 6–8 sentences on the parabola reduction. Draw 5 points on $y = x^2$ and their hull.

Quiz (10 min)

1. Define extreme point. (2 pts)
2. Jarvis time in terms of n and h. (2 pts)

3. Why start at lowest-then-leftmost? (2 pts)
4. Is Jarvis optimal for points in convex position? Why? (2 pts)
5. One sentence: how sorting reduces to hull. (2 pts)

Common mistakes

- Starting at a random point (may be interior; the first “wrap” is undefined).
- Taking the nearest collinear point and cutting a hull edge short.
- Infinite loop because duplicates were not removed (`p` never returns to `start` cleanly, or `q` stays equal to `p`).
- Using angles and hitting the $\pm\pi$ wrap.

Board drawings

1. Rubber band / extreme points.
2. One full Jarvis wrap on 8 points with the candidate ray.
3. Parabola reduction: points `(xi, xi²)` and the lower hull.

Week 5 — Convex hull II (Graham and Andrew)

Time: 75 min lecture + 60 min live coding
Algorithm this week: Andrew’s monotone chain (required), Graham scan (explained)
Board first: sort by x, build lower hull, build upper hull

Timing

Minutes	Do this
0–10	Quiz Week 4
10–30	Graham scan
30–55	Andrew’s monotone chain, invariant, collinear policy
55–65	3D hull preview (no code)
65–75	Graphics: AABB, OBB, culling

Learning goals

1. Explain Graham scan: sort by polar angle, then scan with a stack.
2. Implement Andrew’s algorithm correctly.
3. Handle duplicates and collinear points on the hull.
4. State O(n log n) time and why the scan is O(n) after sorting.
5. Name one 3D hull method and why it is harder.

---

1. Graham scan (20 min)

Steps

1. Find the lowest-then-leftmost point p0.
2. Sort the remaining points by polar angle around p0. Break ties by distance.
3. Scan the sorted list with a stack. While the last three points make a non-left turn, pop.

```
graham(S):
    remove duplicates
    p0 = lowest-then-leftmost
    L = others sorted by (orient(p0, a, b), then dist to p0)
        // a before b if orient(p0,a,b) > 0
        // if collinear, nearer first (they will be popped) or farther first (policy)
    stack = [p0, L[0], L[1]]
    for p in L[2…]:
        while orient(next_to_top, top, p) <= 0: pop
        push p
    return stack
```

`<= 0` pops collinear middles if we drop them.

Why the sort is the dangerous part

Polar sort with `atan2` is fragile. Compare with `orient(p0, a, b)` instead.

If two points are collinear with p0, the nearer one is **not** a hull vertex (under our policy). Sorting nearer-first and using `<= 0` pops it.

Complexity

Sort: O(n log n).
Scan: each point is pushed once and popped at most once, O(n).
Total: O(n log n).

---

2. Andrew's monotone chain (25 min)

This is the algorithm students must implement. It avoids polar angles entirely.

Steps

1. Remove duplicates.
2. Sort points by (x, y).
3. Build the **lower hull** left → right.
4. Build the **upper hull** left → right (or right → left).
5. Concatenate, removing the duplicated endpoints.

```
andrew(S):
    P = unique points sorted by (x, y)
    if |P| <= 2: return P

    lower = []
    for p in P:
        while |lower| >= 2 and orient(lower[-2], lower[-1], p) <= 0:
            pop lower
        push p onto lower

    upper = []
    for p in reverse(P):
        while |upper| >= 2 and orient(upper[-2], upper[-1], p) <= 0:
            pop upper
        push p onto upper

    pop last of lower   // same as first of upper
    pop last of upper   // same as first of lower
    return lower + upper
```

Invariant (lower hull)

The stack is a strictly left-turning chain from the leftmost to the current point. All processed points lie above or on it.

`<= 0` means: pop on right turns **and** on collinear, so we drop middle collinear points.

To **keep** collinear hull points, use `< 0` only.

Why the scan is O(n)

Same stack argument as Graham: each point is pushed once per chain and popped at most once.

Trace on the board

Seven points. Sort. Build lower (3–4 pops). Build upper. Concatenate. Count h.

Andrew vs Graham vs Jarvis

	Jarvis	**Graham**	**Andrew**
Time	Θ(n h)	O(n log n)	O(n log n)
Sort key	none	polar around p0	(x, y)
Implementation risk	infinite loop	polar ties	off-by-one at join
Use when	h is tiny	teaching polar sort	**default in this course**

---

3. 3D preview (10 min)

In 3D the hull is a convex polyhedron.

- **Gift wrapping** still works; a “step” is a face, and the search is over a 1D set of candidate edges. Time grows fast.
- **Incremental:** add a point, remove visible faces, sew the horizon. This is the usual textbook 3D algorithm.
- **Conflict graphs** make expected incremental construction O(n log n).

We will not implement 3D hulls. Graphics engines use them for collision hulls and for silhouette ideas. Students should know the output is a mesh of triangles, not a polygon.

---

4. Graphics connections (10 min)

Structure	How it uses a hull
AABB	hull of the 4 (or 8 in 3D) extreme coordinates; cheaper, looser
OBB	oriented box; a cheap approximation of the 2D hull
Broad-phase collision	if hulls (or AABBs) miss, objects miss
Camera / frustum	a 2D analog is “is this sprite’s hull on screen?”
Shadow / silhouette	extreme points in a light direction (supporting line)

Live sentence: Jarvis is one supporting line after another. A directional extreme query is one step of Jarvis.

Live coding (60 min)

Implement Andrew with the stack drawn.

Draw:

- sorted index next to each point
- lower hull in blue, growing
- upper hull in red, growing
- popped points flashing
- final polygon filled

Step keys as in Week 4.

Then run the same n = 10_000 cloud and circle as Week 4. Andrew should not care that h = n.

Lab

1. Implement Andrew.
2. Compare with last week's Jarvis on:
   - random cloud n = 10_000
   - circle n = 2_000
3. Toggle policy: drop vs keep collinear. Show a 4-point collinear top edge.
4. Unit tests: n = 0,1,2; triangle; all-collinear; square with a point in the middle.

Done when all-collinear returns the two endpoints only (drop policy).

Homework

1. Implement Andrew (or Graham if you justify polar compare without `atan2`).
2. Document the collinear policy in the README.
3. Written: prove the scan is O(n) after sorting (each index pushed/popped ≤ once per chain).
4. Written: one paragraph on AABB vs convex hull as a collision proxy. When is the AABB good enough?

Quiz (10 min)

1. What is the sort key in Andrew? (2 pts)
2. Why is the while-loop total O(n)? (2 pts)
3. What does `orient <= 0` do to collinear hull points? (2 pts)
4. Give one case where Jarvis beats Andrew. (2 pts)
5. Name one 3D hull strategy. (2 pts)

Common mistakes

- Forgetting to unique-sort; then `<= 0` pops everything because consecutive duplicates are collinear with anything.
- Concatenating lower and upper without removing the duplicated endpoints → a zero-length edge and a later crash.
- Building both chains left-to-right with the same orientation test, so one chain turns the wrong way.
- Using `atan2` in Graham and failing the left-half vs right-half.

Board drawings

1. Graham polar sort around p0.
2. Andrew: sorted list, lower stack, upper stack, join.
3. AABB vs true hull around a diagonal stick (AABB is huge; hull is tight).

Week 6 — Line segment intersection (sweep line)

Time: 75 min lecture + 60 min live coding
Algorithm this week: Bentley–Ottmann at teaching level
Board first: a vertical line moving right, active segments sorted along it

---

Timing

Minutes	Do this
0–10	Quiz Week 5
10–20	Naive $O(n^2)$ and why we can do better
20–40	Sweep line, events, status
40–60	Bentley–Ottmann, degeneracy
60–75	Graphics uses; what the lab will simplify

Learning goals

1. State the naive bound and when it is optimal ($\Theta(n^2)$ intersections).
2. Describe a plane sweep: event queue + status.
3. List the event types: left endpoint, right endpoint, intersection.
4. Explain why only **adjacent** segments in the status can be the next intersection.
5. Implement a teaching sweep with a sorted list ($n \le 200$), not a red-black tree.

---

1. Naive algorithm (10 min)

```
for every pair of segments:
    if they intersect: report it
```

Time $\Theta(n^2)$. Space $O(1)$ extra besides the output.

If there are $I = \Theta(n^2)$ intersections, **any** algorithm is $\Omega(n^2)$ just to write them down. Sweep is interesting when I is smaller: maps, UI edges, CAD, almost-planar drawings.

Output-sensitive target: **$O((n + I) \log n)$**.

---

2. The sweep idea (20 min)

Imagine a vertical line L moving from $x = -\infty$ to $+\infty$.

At a typical x, L hits a subset of segments. That subset is the **status**. Order the status by y-coordinate of the hit.

Key lemma (teaching).
Two segments can intersect to the right of L only if they are **neighbors** in the status, or will become neighbors after some event. So we only test neighbors.

Event queue Q

A priority queue of x-coordinates (then y as tie-break):

Event	When	What we do
LEFT	left endpoint of a segment	insert segment into status; test against its two new neighbors
RIGHT	right endpoint	delete segment; test the two neighbors that just became adjacent
INTER	crossing of two segments	report the point; **swap** the two segments in the status; test new neighbor pairs

Store events so each geometric point is unique. If we discover an intersection, we insert an INTER event at that x, unless it is already in Q.

Status T

Ideally a balanced BST ordered by the y-order along L.
For the lab: a sorted array, $O(n)$ insert/delete, fine for $n \le 200$.

Invariant

All intersections with x less than the current event have been reported. The status is the set of segments stabbed by L, ordered by y.

---

3. Bentley–Ottmann, teaching version (20 min)

```
sweep(segments):
    Q = all LEFT and RIGHT endpoints
    T = empty status
    I = []
    while Q is not empty:
        e = pop min x
        if e is LEFT of s:
            insert s into T
            test(s, above(s)); test(s, below(s))
        else if e is RIGHT of s:
            a, b = above(s), below(s)
            delete s from T
            test(a, b)
        else: // INTER of s, t
            report e.point
            swap s and t in T
            test each with its new outer neighbor
    return I

test(s, t):
    if s, t exist and intersect at p with p.x >= current x:
        insert INTER(p, s, t) into Q if new
```

Use Week 2's `segmentsIntersect`. Only `proper` (and, by policy, `touch`) generate INTER events.

Complexity

Each event costs O(log n) with a heap + BST.
Number of events ≤ 2n + I.
O((n + I) log n).

With a list, **O((n + I) n)**, acceptable in lab.

Degeneracy (spend time here)

1. **Vertical segments.** A vertical segment has left = right. Treat it as a special event: insert and delete at the same x, or perturb. Simplest lab policy: reject verticals, or store them with a tiny x-tilt documented in the README.
2. **Several segments meet at one point.** One INTER event with a bundle. Swap the whole bundle reverse-order.
3. **Overlapping collinear segments.** Not a point intersection. Report `overlap` separately; do not put a single INTER in Q.
4. **Endpoint lying on another segment.** `touch`. Decide: report as intersection, split the segment (needed later for DCEL), or ignore. Course default: **report touch, do not split** until Week 8.

---

4. Why the status order is “along L,” not “by endpoint y” (10 min)

Two segments can have left-endpoint y in one order and, after they cross, the opposite order. The status must be the order **at the current x**. After an INTER, we swap.

If a student sorts T by the y of left endpoints and never swaps, they will miss later intersections.

Draw this counterexample and leave it up.

---

5. Graphics (5 min)

- Map overlay (two road networks)
- CAD edge intersections
- Clipping a polyline against a window
- UI: do these two strokes meet?
- Mesh repair: self-intersecting outlines (Week 13 / project)

Live coding (60 min)

Do **not** write a red-black tree.

Visualizer:

- n segments, draggable endpoints
- a vertical sweep line the professor can scrub
- status listed at the right, top to bottom
- events drawn as ticks on the x-axis
- intersections as dots

Script:

1. Two crossing segments — one INTER, a swap, status order flips.
2. Three segments, only two intersections — show that the third pair is never tested while non-adjacent.
3. T-junction — `touch`.
4. AABB-overlap, no intersection — no event.

Implement the list-based sweep live if time; otherwise scrub a prepared demo and implement the event loop together.

Lab

1. Input: list of segments. Output: all `proper` and `touch` points, drawn.

2. Use a sorted list for T and a binary heap or sorted array for Q.
3. $n \le 200$. Brute-force pairs as an oracle: the two outputs must match (as sets of points, with an epsilon).
4. Reject or specially handle verticals; document the choice.

Done when the oracle and the sweep agree on a random 80-segment instance.

Homework

1. Written: why only neighbors in T need to be tested. 1 page, with the "cross only if they become adjacent" picture.
2. Written: why status order is the y-order along L, not endpoint y. Use the crossing-pair counterexample.
3. Code: teaching sweep + oracle test.

Quiz (10 min)

1. Naive time? Sweep time in n and I (BST version)? (2 pts)
2. Name the three event types. (2 pts)
3. After an INTER, what happens in T? (2 pts)
4. If $I = n^2/4$, is sweep asymptotically better than naive? (2 pts)
5. Why can a list replace a BST in the lab? (2 pts)

Common mistakes

- Testing every pair still, and calling it a sweep because a line is drawn.
- Not swapping at INTER.
- Inserting the same intersection twice and looping.
- Using `touch` as `none`.
- Vertical segments crashing the x-order.

Board drawings

1. Sweep line, three active segments, T listed by y.
2. Event timeline: LEFT, LEFT, INTER, RIGHT, …
3. Two segments that cross, status before and after swap.

Week 7 — Polygon triangulation

Time: 75 min lecture + 60 min live coding
Algorithm this week: ear clipping
Board first: a concave polygon, one ear shaded, then the remaining polygon

Also today: hand out the **midterm topic list** (end of this note).

Timing

Minutes	Do this
0–10	Quiz Week 6
10–25	Why triangulate; $n - 2$ triangles
25–50	Ears and ear clipping
50–65	Monotone polygons (idea)
65–75	Constrained vs mesh triangulation; midterm list

Learning goals

1. Prove a simple polygon with n vertices has $n - 2$ triangles and $n - 3$ diagonals.
2. Define an ear and Meisters' theorem (teaching statement).
3. Implement ear clipping and know it is $O(n^2)$.
4. Explain why a y-monotone polygon is easier ($O(n)$ after sort).
5. Distinguish polygon triangulation from Delaunay (Week 11).

---

1. Why triangulate (15 min)

GPUs draw triangles. A filled polygon in a canvas, a font outline, a UI blob, a roof footprint — all become triangles.

Theorem. Every simple polygon with $n \geq 3$ vertices has a triangulation: $n - 2$ triangles, $n - 3$ diagonals, using only existing vertices.

Proof sketch (induction).

$n = 3$: already a triangle.

$n > 3$: a simple polygon has a diagonal (exists; we will get one from an ear). A diagonal splits P into P_1, P_2 with $n_1 + n_2 = n + 2$ vertices (the two endpoints are shared). By induction, triangles = $(n_1 - 2) + (n_2 - 2) = n - 2$.

Existence of a diagonal. Every simple n > 3 polygon has at least three convex vertices. At least one of those is an **ear**: the diagonal between its neighbors lies inside P. Clipping that ear is a diagonal.

Do not spend the whole lecture on the existence proof. Draw it. Assign the write-up as homework.

---

2. Ears (25 min)

Vertex vi is an **ear tip** if:

1. vi is convex (not reflex, not a skip of a flat-only policy),
2. the diagonal `v_{i-1} v_{i+1}` lies **inside** P,
3. equivalently: triangle `v_{i-1} v_i v_{i+1}` contains **no other vertex** of P.

Condition 3 is what we implement. Because P is simple, "no vertex inside the ear triangle" plus "vi convex" implies the diagonal is inside.

Meisters' theorem (state)

Every simple polygon with n ≥ 4 has at least two ears.

Ear clipping

```
earClip(P):
    if not simple: fail
    V = cyclic list of vertices
    T = []
    while |V| > 3:
        found = false
        for each vi in V:
            if isEar(vi, V):
                T.append(triangle(v_{i-1}, vi, v_{i+1}))
                remove vi from V
                found = true
                break
        if not found: fail   // not simple, or numeric garbage
    T.append(remaining triangle)
    return T

isEar(vi, V):
    if orient(v_{i-1}, vi, v_{i+1}) is not a convex turn: return false
    for each vertex q in V except the three:
        if q is inside triangle(v_{i-1}, vi, v_{i+1})
           or on its boundary (except the three vertices):
            return false
    return true
```

Use Week 2 point-in-triangle: three orientation tests of the same sign (plus boundary policy).

Complexity

Each of n − 3 clips may scan O(n) vertices to test ears.
$O(n^2)$. Fine for n up to a few thousand (UI shapes, font glyphs). Not fine for a 200k-vertex GIS lake.

Faster: O(n log n) via monotone subdivision (below). O(n) exists (Chazelle) — mention, do not teach.

Degeneracy

- Holes: ear clipping as written does **not** handle holes. Either ban holes or cut a channel to the outer boundary first.
- Flat vertices: treat as not ears; they can be skipped or removed in a preprocess.
- Almost-collinear "no vertex inside" with a vertex sitting on the diagonal: reject as ear (boundary case).

---

3. Monotone polygons (15 min)

A polygon is **y-monotone** if its boundary splits into two chains from the top vertex to the bottom vertex, and each chain is never-upward (or never-downward).

A y-monotone polygon can be triangulated in **O(n)** with a stack, similar in spirit to Andrew's hull scan.

Full O(n log n) pipeline:

1. Add diagonals to split a simple polygon into y-monotone pieces (sweep, O(n log n)).
2. Triangulate each piece in linear time.

Teach step 1 as a picture: merge/split/start/end/regular vertices. Do **not** require the sweep implementation. Students should be able to **label** a vertex as split vs merge on a drawing.

Vertex type	Local picture	Action (idea)
Start	both neighbors below, interior below	new helper
End	both neighbors above, interior above	close a piece
Split	both neighbors below, interior above	add diagonal to helper
Merge	both neighbors above, interior below	add diagonal later
Regular	one neighbor up, one down	update helper

This table is enough for the midterm at the level "what is a split vertex?"

---

4. Two different triangulations (10 min)

	Polygon triangulation	Delaunay (Week 11)
Input	a simple polygon (edges are constraints)	a point set
Edges	all boundary edges must appear	no boundary unless we add a hull
Quality	any triangulation is allowed	empty circumcircle
Use	fill a shape	well-shaped mesh, terrain

Constrained Delaunay sits between them: respect given edges, maximize the Delaunay property elsewhere. Name it. Project option.

Live coding (60 min)

Ear clipping in step mode.

- Current candidate vertex highlighted
- Ear triangle filled green if legal, red if a point is inside
- Clipped ears stay as faint triangles
- Remaining polygon outlined

Clip a C-shaped polygon. Students should see a reflex vertex refused, then accepted after its neighbor is removed.

Fail on a bowtie with a clear error, not an infinite loop.

Lab

1. Implement `earClip`.
2. Draw diagonals and number triangles in clip order.
3. Inputs: convex, C-shape, a 12-vertex simple room.
4. Bowtie must throw.

Done when the C-shape produces $n - 2$ triangles and the union visually fills the polygon.

Homework

1. Implement ear clipping.
2. Written: induction that a simple n-gon has $n - 2$ triangles.
3. Written: define an ear. Give a polygon with exactly two ears (a convex quadrilateral is too easy; use a convex n-gon — it has n ears — so instead use a concave quad, which has two).

4. **Study** the midterm list below.

Quiz (10 min)

1. How many triangles in a simple 10-gon? How many diagonals? (2 pts)
2. Define ear tip. (3 pts)
3. Ear clipping time? (2 pts)
4. Does ear clipping as taught handle holes? (1 pt)
5. Name one difference vs Delaunay. (2 pts)

Midterm topic list (print / post today)

Week 8, written, 60–75 minutes, no laptop.

1. `orient`, signed area, predicates vs constructions
2. Segment intersection types: proper / touch / overlap / none
3. Point in polygon: even–odd and the vertex-hit rule
4. Convex set; convex vs simple vs self-intersecting
5. Same-turn ⇔ convex for **simple** polygons (short proof)
6. Jarvis: idea, Θ(n h), when it is slow
7. Andrew: sort key, stack, O(n log n), collinear policy
8. Hull lower bound via the parabola (idea)
9. Sweep: events, status, why neighbors only
10. Ear: definition, $n - 2$ triangles, $O(n^2)$
11. One degeneracy question (collinear / T-junction / ray through vertex)

No Voronoi, no kd-tree, no DCEL on the midterm.

Common mistakes

- Testing `isEar` without the convex-turn test (a reflex “ear” diagonal is outside).
- Forgetting the polygon is cyclic.
- Using point-in-polygon on the whole P instead of point-in-triangle.
- Infinite loop on a self-intersecting input.
- Claiming any triangulation is Delaunay.

Board drawings

1. Induction split along a diagonal.
2. One ear, one interior point that invalidates a candidate.
3. Split / merge vertex sketches.
4. Same point set: a bad skinny triangulation vs a Delaunay preview.

Week 8 — Midterm and DCEL

Time: 60–75 min midterm, then 30–45 min lecture
New structure: doubly-connected edge list
Lab: walk a face

No homework this week except keeping midterm notes.

Midterm (60–75 min)

Written. No laptop. 100 points. Topic list was issued in Week 7.

Suggested paper (edit names/numbers as you like)

Q1. Predicates (16 pts)
A=(0,0), B=(4,1), C=(2,3), D=(2,0).

1. Sign of `orient(A,B,C)`? (4)
2. Type of intersection of AB and CD? (4)
3. Is "circumcenter of ABC" a predicate or a construction? (4)
4. Give one reason not to use `atan2` for left-of-line. (4)

Q2. Polygons (16 pts)
1. Define convex set. (4)
2. Classify: convex hexagon, C-shape, bowtie. (6)
3. Short proof: simple + all turns the same sign ⇒ locally convex, and why the bowtie shows simplicity is required. (6)

Q3. Hulls (24 pts)
1. Jarvis time in n, h. Worst-case input. (6)
2. Andrew: write the lower-hull while-condition and say what `<= 0` does. (8)
3. Parabola reduction in 4–6 sentences. (10)

Q4. Sweep (20 pts)
1. Three event types. (6)
2. Why test only status neighbors? (8)
3. After INTER, what happens to the two segments in T? (6)

Q5. Triangulation and degeneracy (24 pts)
1. Number of triangles and diagonals in a simple 12-gon. (6)
2. Define an ear. (6)
3. Ray from q hits a polygon vertex. What goes wrong, and what is the half-open fix? (6)
4. T-junction: `proper`, `touch`, `overlap`, or `none`? (6)

Grading note

Award partial credit for a correct picture with a wrong name. Do not award credit for “O(n log n)” on Jarvis without h.

Lecture — DCEL (30–45 min)

Why a new data structure

Arrays of vertices are enough for a single polygon. They fail as soon as we have:

- many faces (a triangulation, a map, a mesh)
- the need to walk around a face
- the need to walk around a vertex
- the need to split an edge at an intersection

A **doubly-connected edge list** stores a planar subdivision.

Graphics people already know this as a **half-edge mesh**.

Records

Vertex

- `x, y`
- `incidentEdge` — one outgoing half-edge

Half-edge

- `origin` — vertex
- `twin` — the opposite half-edge
- `next` — next half-edge around the face
- `prev` — previous around the face (optional if you always have `next` and `twin`)
- `face` — the face on the left (for CCW faces)

Face

- `outer` — one half-edge on the outer boundary, or null if unbounded
- `inners` — half-edges on hole cycles, if any

Convention for this course: **walk `next` so the face is on the left** (CCW outer boundary, CW holes).

```
e.twin.origin == e.next.origin
e.twin.twin == e
e.next.prev == e
```

Pictures (draw both)

1. One triangle: 3 vertices, 3 interior half-edges, 3 outer (unbounded-face) half-edges, 2 faces.
2. Two triangles sharing an edge: 4 vertices, 10 half-edges (5 edges × 2), 3 faces (two bounded + unbounded).

Count with the students. If the counts mismatch, the DCEL is wrong.

Operations we need

```
walkFace(e):
    start = e
    do:
        visit e
        e = e.next
    while e != start

walkVertex(e):          // outgoing edges around origin
    start = e
    do:
        visit e
        e = e.twin.next
    while e != start
```

Splitting an edge at a point p (needed after Week 6 intersections, map overlay, mesh repair):

1. Insert vertex p.
2. Replace one edge by two.
3. Update `next` / `twin` on both faces.

Do this as a diagram. Coding the split is optional extra, not the lab.

Graphics connection

DCEL idea	Engine name
half-edge	half-edge / winged-edge
`next` around face	face loop
`twin.next` around vertex	vertex ring
faces	mesh faces / materials
split edge	edge split, subdivision

Three.js `BufferGeometry` is **not** a DCEL. It is a triangle soup (or indexed soup). That is why adjacency queries are painful and why mesh-repair tools rebuild a half-edge structure first.

Live coding (if any time remains)

Draw a hardcoded DCEL for two triangles. Log `walkFace` for each bounded face and `walkVertex` for the shared vertices.

Otherwise do this as the first 20 minutes of lab.

Lab

Give students a JSON DCEL for:

- one triangle
- two triangles sharing an edge
- a convex pentagon with one diagonal (three faces)

They write:

1. `walkFace(edgeId) -> [vertexIds]`
2. `walkVertex(edgeId) -> [neighborVertexIds]`
3. A check: every half-edge's twin's twin is itself; every face walk returns to start.

Done when the pentagon-with-diagonal reports two bounded faces with the correct vertex cycles.

Homework

None. Optional: read de Berg, chapter on DCEL (short).

Quiz

None this week.

Common mistakes

- Storing only one directed edge per segment, then being unable to walk both faces.
- Mixing CW and CCW in the same file.
- Forgetting the unbounded face. Euler numbers will not add up.
- Treating Three.js triangle indices as a DCEL in the lab report.

Board drawings

1. Half-edge with arrows: `origin`, `twin`, `next`, `face`.

2. Two triangles, every half-edge labeled.
3. `walkVertex` = `twin.next` loop.

Week 9 — Point location and range search

Time: 75 min lecture + 60 min live coding
Algorithm this week: 2D kd-tree range query
Board first: a point set, a query rectangle, brute force vs pruned boxes

---

Timing

Minutes	Do this
0–5	Return midterms briefly; no quiz
5–20	Point location: slabs, Kirkpatrick (idea)
20–45	kd-trees and range search
45–60	Quadtrees and range trees (survey)
60–75	BVH as the graphics version

Learning goals

1. State the point-location problem and the slab method's cost.
2. Build a 2D kd-tree and run an axis-aligned range query.
3. Explain prune vs visit using the node's bounding box.
4. Contrast kd-tree, quadtree, range tree, and BVH.
5. Connect range search to picking and frustum culling.

---

1. Point location (15 min)

Problem. Preprocess a planar subdivision (a DCEL). Then, for a query point q, report the face that contains q.

Slab method

Draw a vertical line through every vertex. The plane splits into slabs. Inside a slab the upper/lower edge order is constant. Binary search the slab by x, then binary search the edges by y.

- Query: $O(\log n)$
- Space: $O(n^2)$ in the worst case (every pair of lines contributes)

Too much space. Mention as the "obvious" structure.

Kirkpatrick's hierarchy (idea only)

Repeatedly remove an independent set of low-degree vertices from a triangulation and retriangulate. A query walks from the coarsest triangle down to the original face.

- Query: O(log n)
- Space: O(n)

We will not implement this. Students should remember: **point location is not "test every face."**

Practical substitute in this course

For a triangulation or a UI scene, a **BVH or kd-tree on bounding boxes** plus a point-in-polygon / point-in-triangle test is what they will actually ship.

---

2. kd-trees (25 min)

Problem. Preprocess n points. Query: report (or count) points inside a rectangle R = [x1, x2] × [y1, y2].

Build

Alternate the splitting axis: depth 0 splits on x at the median, depth 1 on y, and so on.

```
build(points, depth):
    if points is empty: return null
    if |points| == 1: return leaf(points[0])
    axis = depth % 2          // 0 = x, 1 = y
    sort points by axis
    mid = median
    return node(
        axis, points[mid],
        build(left half, depth+1),
        build(right half, depth+1)
    )
```

Build time $O(n \log^2 n)$ with a sort at each node, or O(n log n) if we presort both axes. Space O(n).

Each node implicitly owns an axis-aligned region (the cell). Store the cell or recompute it from the path.

Range query

```
query(node, R):
    if node is null: return
    if node.cell is disjoint from R: return          // prune
    if node.cell is inside R: report all points in subtree  // take
    else:
        if node.point is in R: report it
        query(node.left, R)
        query(node.right, R)
```

Worst-case query for reporting in 2D is $O(\sqrt{n} + k)$, where k is the number reported. (The $\sqrt{n}$ comes from a thin query that stabs many cells.) Average random queries are much faster.

Trace

Eight points. Build the tree on the board. Query a rectangle that misses the right subtree entirely. Cross out that subtree.

---

3. Quadtrees and range trees (15 min)

Quadtree

Split the **space** into four squares, not the point set by median. Good for uniformly spread points and for images / tiles. Degenerates if all points sit in one corner (need a stop rule: capacity, or minimum cell size).

Graphics: mip-like spatial bins, terrain tiles, “what is in this screen tile?”

Range tree

A tree on x; each node stores a tree of its subtree on y.
Query: $O(\log^2 n + k)$. Space: $O(n \log n)$.

Name it as the theoretically nicer range-search structure. Do not implement.

---

4. BVH — the graphics kd-tree (15 min)

A **bounding volume hierarchy** is a binary tree of objects (triangles, meshes), not of points.

- Each leaf: one object (or a small bucket)
- Each node: an AABB (or OBB / sphere) of its descendants
- Query (ray, frustum, click): skip a node if the query misses its box

This is Week 2’s AABB reject, recursively.

	kd-tree	BVH
Splits	space, median of points	objects into two groups
Bounds	implicit cells	stored AABB
Typical query	orthogonal range	ray / frustum
Used in	databases, GIS, this lab	Three.js, game engines, `three-mesh-bvh`

Week 13 will pick a triangle with a BVH. Today, say the word and draw one tree of three AABBs.

Live coding (60 min)

Build a kd-tree on a clickable point set.

Query tool: drag a rectangle.

Draw:

- splitting lines (x blue, y red)
- visited nodes in orange
- pruned cells in gray
- reported points in green
- counters: visited, pruned, reported, brute-force checks

Compare with brute force on 5_000 points. The prune count should be the story, not the milliseconds alone.

Lab

1. Implement `buildKdTree` and `rangeQuery`.
2. Brute-force oracle: same reported set.
3. n = 5_000 random points, 20 random rectangles. Print average visited vs n.
4. One handwritten figure in the README: 8 points, the tree, one query, the pruned subtree circled.

Done when the oracle always matches and at least one query prunes ≥ 40% of points.

Homework

1. Hand-draw a kd-tree for these points:
   (2,3), (5,4), (9,6), (4,7), (8,1), (7,2), (6,3), (3,6).
   Split x at the median first. Show a query [3,7] × [2,5] and mark prune / visit.
2. Written: O(√n + k) in one paragraph — a thin vertical query that hits many vertical splits.

3. Written: one use of a BVH in a Three.js app.

Quiz (10 min)

1. Slab method: query time and the space problem. (2 pts)
2. kd-tree: which axis at depth 3? (2 pts)
3. When do we prune a node? (2 pts)
4. Range tree vs kd-tree: one advantage each. (2 pts)
5. BVH stores what at each node? (2 pts)

Common mistakes

- Splitting always on x.
- Forgetting to test the point stored at an internal node.
- Pruning with the wrong cell (using the child's cell for the parent).
- Reporting points that lie on the boundary twice, or dropping boundary points. **Policy:** R is closed; include the boundary.
- Building a quadtree and calling it a kd-tree in the report.

Board drawings

1. Slabs through vertices.
2. kd-tree splits on 8 points.
3. A query rectangle vs a node cell: disjoint / contained / overlap.
4. BVH of three triangles.

Week 10 — Voronoi diagrams

Time: 75 min lecture + 60 min live coding
What we implement: discrete (pixel) Voronoi + nearest-site query
What we do not implement: Fortune's algorithm
Board first: three sites, three perpendicular bisectors, one Voronoi vertex

---

Timing

Minutes	Do this
0–10	Quiz Week 9
10–30	Definition, cells, vertices, empty circle
30–50	Fortune at intuition level
50–65	Applications
65–75	Discrete Voronoi and the dual teaser

Learning goals

1. Define the Voronoi cell of a site.
2. State the empty-circle property of a Voronoi vertex.
3. Describe Fortune's beach line and the two event types.
4. Explain a discrete Voronoi (for each pixel, nearest site).
5. Name three applications: nearest neighbor, territory, coverage.

---

1. Definition (20 min)

Let S = {s1, …, sn} be sites in the plane (assume general position: no four cocircular, no three collinear — then mention what fails).

The **Voronoi cell** of si is

```
V(si) = { p | dist(p, si) ≤ dist(p, sj) for all j }
```

The **Voronoi diagram** VD(S) is the set of points that have at least two nearest sites (the boundaries) together with the cells.

Geometry of an edge

The set of points equidistant from si and sj is the **perpendicular bisector** of si sj.
A Voronoi edge is a (possibly unbounded) piece of that bisector: the points that are closer to {si, sj} than to any other site.

Geometry of a vertex

A Voronoi vertex is equidistant from **three** sites (usually). It is the **circumcenter** of those three sites.

Empty-circle property.
The circle through those three sites contains **no site in its interior**. If it did, that site would be closer to the center, and the vertex would not be Voronoi.

This sentence is the homework proof and the bridge to Delaunay next week.

Unbounded cells

A cell is unbounded iff its site lies on the convex hull of S.
Draw a hull and show rays going to infinity.

Complexity

For n sites in 2D: O(n) vertices and edges. The diagram is a linear-size planar graph. That is why it is usable.

---

2. Fortune's algorithm — intuition only (20 min)

A sweep line from top to bottom (standard picture).

Behind the sweep line the diagram is not yet safe: a site just below the line could still steal territory.

The **beach line** is the boundary between the "decided" region and the undecided region. It is a sequence of parabolic arcs. Each arc is the set of points equidistant from a site and the sweep line.

Events

Event	Meaning	Effect
Site event	sweep line hits a new site	a new arc appears on the beach line
Circle event	sweep line hits the bottom of a circle through three sites that make consecutive arcs	an arc disappears; a Voronoi vertex is born

Data structures (name only): a balanced tree for the beach line, a priority queue for events. Time O(n log n).

Do not implement this in the lab. Draw the beach line three times: before a site event, after a site event, at a circle event.

If students ask for code, point to a reference implementation and to Week 11: we will build the dual instead.

---

3. Applications (15 min)

Application	How
Nearest neighbor	the cell that contains q; or walk / point-locate in VD
Territory / service areas	cell = region assigned to a facility
Robot coverage / paths	Voronoi edges are locally farthest from sites (obstacles)
Stippling / mosaics	sites = dots; cells = tiles (project)
Meteorology / GIS	Thiessen polygons (same object)
Procedural cities	cells → blocks, dual Delaunay → roads (project)

Live demo idea: move q, color the nearest site. That *is* the Voronoi membership test, even if we compute it by brute force this week.

---

4. Discrete Voronoi (10 min)

For each pixel p, color p by argmin_i dist(p, si).

This is O(pixels × n). Fine for a 600×400 canvas and n ≤ 40. Jump-flooding and cone-shader methods exist on the GPU; mention for IGWT students, do not require.

The discrete picture is a raster approximation. Vertices will look like pixels where three colors meet.

Next week we get the exact combinatorial diagram via Delaunay.

Live coding (60 min)

1. Click to add sites.
2. For each pixel (or a coarse grid), color by nearest site (semi-transparent).
3. Overlay sites as dots.
4. A draggable query point q; draw the segment to its nearest site; print the site id.
5. Optional: draw perpendicular bisectors of all pairs (messy) to motivate “we need a better algorithm.”

Do **not** start Fortune.

If you have a small Delaunay library already, you may overlay the exact Voronoi as a teaser and say “dual, next week.”

Lab

1. Discrete Voronoi on a canvas.
2. n sites, add/remove/drag.
3. Query point with nearest-site highlight.
4. Screenshot: 8 sites, query in a cell, and a place where three colors meet (approximate vertex).
5. Written in README: why a vertex is a circumcenter, in your own words.

Done when dragging a site updates the coloring live.

Homework

1. **Proof (required).** Let v be a Voronoi vertex defined by sites a, b, c. Show that the circumcircle of abc contains no other site in its interior. (Use the definition of V(·) and dist(v,a) = dist(v,b) = dist(v,c).)
2. Written: which cells are unbounded, and why.
3. Written: Fortune's two event types, three sentences each.
4. No Fortune code.

Quiz (10 min)

1. Define V(si). (2 pts)
2. A Voronoi vertex is the _____ of three sites. (2 pts)
3. Empty-circle property, one sentence. (2 pts)
4. Site event vs circle event. (2 pts)
5. A cell is unbounded iff the site is _____. (2 pts)

Common mistakes

- Drawing the Delaunay triangulation and calling it Voronoi (they are dual; edges are not the same).
- Thinking every pair of sites produces a Voronoi edge (only neighbors do).
- Implementing Fortune badly and spending the week in debugging. That is why we forbid it.
- Using `==` on distances; use `≤` and a consistent tie-break (smaller site index).

Board drawings

1. Three sites, three bisectors, one vertex, empty circle.
2. Hull sites with unbounded rays.
3. Beach line: sweep line, two parabolas, a new site punching an arc.
4. Dual teaser: connect sites whose cells share an edge — that is next week.

Week 11 — Delaunay triangulation

Time: 75 min lecture + 60 min live coding
Algorithm this week: incremental insertion + legalize / edge flip
Board first: two adjacent triangles, one illegal edge, the flip

---

Timing

Minutes	Do this
0–10	Quiz Week 10
10–25	Dual of Voronoi; empty circumcircle
25–50	Legal / illegal edges and flips
50–65	Incremental insertion (Bowyer–Watson teaching level)
65–75	Constrained Delaunay; graphics uses

Learning goals

1. Define the Delaunay triangulation via the empty circumcircle.
2. State the duality with the Voronoi diagram.
3. Test an edge for legality and flip it.
4. Insert a point and restore the Delaunay property.
5. Know what a constrained Delaunay triangulation is for.

---

1. Definition and dual (15 min)

A triangulation T of a point set S (plus the hull edges) is **Delaunay** if every triangle's circumcircle contains **no site of S in its interior**.

Duality.
Connect two sites by a Delaunay edge iff their Voronoi cells share an edge.
A Delaunay triangle corresponds to a Voronoi vertex (the circumcenter).
Unbounded Voronoi cells correspond to hull sites; hull edges are Delaunay.

So: Week 10's empty circle at a Voronoi vertex **is** the empty circumcircle of a Delaunay triangle.

Why we care. Among all triangulations of S, the Delaunay triangulation maximizes the minimum angle (no skinnier triangle can be improved by a flip). That is the right default mesh for terrain and interpolation. It is **not** always the minimum-weight (shortest total edge length) triangulation. Do not claim that.

2. Legal edges and flips (25 min)

Consider two triangles abd and acd that share edge ad.
(Or abc and adc — pick one labeling and stick to it.)

Edge ad is **illegal** if the circumcircle of one triangle contains the opposite vertex of the other.

Incircle test (predicate).
For triangle abc (CCW) and point d:

```
incircle(a, b, c, d) > 0  ⇒  d is inside the circumcircle of abc
```

The exact 4×4 determinant is in de Berg / Shewchuk. For the lab, a geometric construction is acceptable if you invert a well-tested circumcenter and compare distances — but **state that it is unstable**. Prefer the determinant if you can.

```
// teaching form: compare angles
// ad is illegal iff angle at b + angle at c > 180°
// (the two angles opposite the shared edge)
```

The angle test is the one to draw: if the two angles opposite ad sum to more than 180°, flip.

Flip.
Replace diagonal ad by the other diagonal bc. The two triangles become abc and bdc (relabel to match your figure).

Theorem (teaching).
A triangulation is Delaunay iff every interior edge is legal.
Repeatedly flipping illegal edges terminates at the Delaunay triangulation (each flip increases the min angle; there are finitely many triangulations).

```
legalize(edge ad):
    if ad is a hull edge: return
    let abc, adc be the two triangles
    if incircle(a, b, c, d) > 0:   // d inside abc, or the symmetric test
        flip ad → bc
        legalize(ab); legalize(ac)  // new edges may be illegal
        // (the exact pair is the edges of the new triangles that are not the flip)
```

---

3. Incremental insertion (15 min)

Bowyer–Watson, teaching version.

Maintain a Delaunay triangulation of the points inserted so far. Start with a huge bounding triangle that contains S.

```
insert(p):
    find the triangle t that contains p          // walk or walk+barycentric
    if p is inside t:
        split t into three triangles
        legalize the three new interior edges
    if p is on an edge e:
        split the one or two triangles of e into two each
        legalize the new edges
```

Locate t. For $n \le 80$, test every triangle (point-in-triangle from Week 2 / 7). For a nicer demo, walk from a random triangle toward p (good in practice, worst-case linear).

Bowyer–Watson cavity form (picture).
Delete every triangle whose circumcircle contains p. The hole is a star-shaped polygon. Connect p to every boundary vertex. That is equivalent to split-and-legalize.

Complexity: expected $O(n \log n)$ with a good location structure; a naive lab is $O(n^2)$, which is fine for $n \le 80$.

---

4. Constrained Delaunay and graphics (10 min)

A **constrained Delaunay triangulation (CDT)** must include a given set of edges (a polygon boundary, a river, a road). Edges that are not constraints still satisfy a constrained empty-circle property.

Use: triangulate a polygon *and* keep it well-shaped; game navmeshes; GIS.

Use	Why Delaunay
Terrain	interpolate height; avoid slivers
Remeshing	better triangles before shading
Cloth / simulation	more stable elements
Lightmap packing	well-distributed vertices
Procedural cities	dual of Voronoi cells

Week 7's ear clipping can produce slivers. Show one picture: same polygon, ear-clip vs CDT.

Live coding (60 min)

$n \le 40$ points, click to insert.

Draw:

- triangles
- circumcircle of the triangle under the mouse

- illegal edges in red
- a flip as an animation (old diagonal fades, new one appears)

Script:

1. Four points, one illegal diagonal, one keypress flip.
2. Insert a fifth point inside a triangle, three legalize calls.
3. Show a circumcircle that contains a point — students should shout "illegal."

Lab

1. Implement `incircle` (or the angle-sum test) and `flip`.
2. Given a triangulation (JSON), legalize until none remain. Draw before/after.
3. Incremental insert for $n \leq 80$ with visible flips.
4. Super-triangle: clip it from the final display.

Done when a known 6-point example matches a reference screenshot (provide one).

Homework

1. Implement edge flip + legalize.
2. Written: prove that a Voronoi vertex's empty circle is the empty circumcircle of the dual triangle (connect Week 10 homework to today).
3. Written: why Delaunay is not necessarily minimum-weight. One counterexample sketch is enough (a very flat quad where the short diagonal is illegal — actually be careful: in a quad the Delaunay diagonal is the one that satisfies incircle, which is the one that maximizes min angle, not always the shorter). State this honestly: **shorter ≠ Delaunay**.
4. One paragraph: CDT vs ear clipping for a game navmesh.

Quiz (10 min)

1. Empty circumcircle property. (2 pts)
2. Dual: a Delaunay edge corresponds to what in VD? (2 pts)
3. When is a shared edge illegal? (2 pts)
4. What does a flip replace? (2 pts)
5. Does Delaunay always minimize total edge length? Yes/no. (2 pts)

Common mistakes

- Flipping a hull edge.
- Incircle with the wrong orientation (sign flips; force CCW before the test).
- Forgetting to legalize recursively after a flip.
- Leaving the super-triangle in the mesh used for area/terrain.
- Calling ear clipping “Delaunay.”

Board drawings

1. Dual: VD in dashed lines, DT in solid, one circle.
2. Illegal edge and the flip, with both circumcircles.
3. Insert p, cavity, retriangulate to p.
4. Skinny ear-clip triangle vs a flipped Delaunay pair.

Week 12 — Closest pair, arrangements, Minkowski sums, visibility

Time: 75 min lecture + 60 min live coding
Required algorithm: closest pair of points, divide and conquer
Survey only: arrangements, Minkowski sums, visibility graphs
Board first: split by x-median, two recursive pairs, the strip

---

Timing

Minutes	Do this
0–10	Quiz Week 11
10–40	Closest pair
40–55	Survey: arrangements, Minkowski, visibility
55–70	What we are not teaching, and why you still need the names
70–75	Project menu (point to Week 14–15 notes)

Learning goals

1. State the closest-pair problem and the naive $\Theta(n^2)$ algorithm.
2. Run the $O(n \log n)$ divide-and-conquer algorithm, including the strip.
3. Explain why only a constant number of strip points can lie in a $\delta \times \delta$ box.
4. Define an arrangement, a Minkowski sum, and a visibility graph at the level of one picture each.
5. Match each survey topic to one graphics / robotics use.

---

1. Closest pair (30 min)

Input: n points.
Output: a pair p, q minimizing dist(p, q). Assume distinct points.

Naive: all pairs, $\Theta(n^2)$.

Algorithm

Presort by x into Px and by y into Py (or sort y inside the recursion).

```
closest(Px, Py):
    if n <= 3: return brute force
    split Px into L and R at the median x = mid
    split Py into Ly, Ry (points of L / R, still sorted by y)
    (dL, pairL) = closest(L, Ly)
    (dR, pairR) = closest(R, Ry)
    δ = min(dL, dR)
    pair = the winner
    strip = points of Py with |x − mid| < δ
    scan strip in y-order:
        for each point, compare to the next few points
            (while Δy < δ; at most ~7 in theory)
        if a closer pair is found: update δ and pair
    return (δ, pair)
```

The strip argument (this is the lecture)

Any pair closer than δ cannot have both points in L or both in R (those were already checked). So both sides of the median line, and both in the 2δ-wide strip.

In the strip, sort by y. For a point p, only points q with `q.y − p.y < δ` can beat δ. Those points lie in a δ × 2δ rectangle. Packing: disks of radius δ/2 around strip points are disjoint and sit in a box that holds only a constant number (classically 6–8, depending on the write-up). So the inner loop is O(1) per point, the merge is O(n), and

```
T(n) = 2 T(n/2) + O(n) = O(n log n)
```

Draw the packing. Do not handwave “we only check 7” without the box.

Implementation notes

- Presort; do not sort the whole set in every recursive call.
- Build the strip from the already-sorted Py in linear time.
- Ties: any closest pair is fine; report one.
- Duplicates: distance 0; reject or treat as a special case in the project, not here.

---

2. Survey (15 min)

Each topic: definition, one picture, one complexity sentence, one use. No code.

Line arrangements

n lines divide the plane into faces, edges, vertices.

- Complexity: $\Theta(n^2)$ faces, edges, vertices.
- **Zone theorem (name):** the zone of one line (faces it touches) has complexity O(n).
- Incremental construction of an arrangement is $O(n^2)$.

- Use: visibility in a line-like world, duality (points ↔ lines), some graphics papers on line-space.

We will not build one.

Minkowski sums

```
A ⊕ B = { a + b | a ∈ A, b ∈ B }
```

If A is an obstacle and B is a robot (as a set of vectors from a reference point), then $A \oplus (-B)$ is the **configuration-space obstacle**: the robot (as a point) cannot enter it.

For two convex polygons, the Minkowski sum is a convex polygon whose edges are the merged edge-direction lists. Time $O(n + m)$.

Use: 2D collision ("does the character's shape hit the wall?"), offset curves, packing.

SAT (separating axis theorem) in games is a cousin: convex vs convex.

Visibility graphs

Vertices: corners of polygonal obstacles (plus start and goal).
Edge: if the segment between two vertices does not stab an obstacle.

Shortest path among obstacles = shortest path in this graph (2D, polygonal).
Size: $O(n^2)$ in the worst case.
Use: crowd / robot project; navmesh is the modern games substitute.

Voronoi roadmaps (Week 10) stay away from obstacles; visibility graphs hug corners. Different aesthetics, both valid project bases.

---

3. The map of the rest of the field (15 min)

Write this table and leave it up through the project.

Name	We did?	If you need it later
Predicates, intersection, PIP	yes	kernel of everything
Hulls	yes	collision, culling
Sweep intersections	yes	CAD, maps
Ear clipping	yes	UI, fonts
DCEL	yes	meshes
kd-tree / BVH	yes	picking
Voronoi / Delaunay	yes	terrain, regions
Closest pair	today	proximity
Fortune fully	no	library
Kirkpatrick	no	library
Arrangements	name	reading
Minkowski	name	collision project
Visibility graph	name	path project
3D Delaunay	no	meshing tools
Exact kernels	Week 13	Shewchuk

Live coding (60 min)

Closest pair with the strip drawn.

- Points
- Median line
- Recursive bounding boxes (optional)
- Strip in yellow
- Candidate pairs in the strip as thin lines
- Current best pair thick

Keys: step the merge scan. Students should see that most strip points compare to 1–3 neighbors, not to everyone.

Then run n = 2_000 vs brute force. Times should diverge.

Lab

1. Implement closest pair, O(n log n).

2. Brute-force oracle for $n \le 800$.
3. Random n = 2_000, 5_000: print time vs brute (brute only at 2_000 if slow).
4. One trace on a 12-point set in the README: δ from left, δ from right, strip members, final pair.

Done when oracle matches and the strip scan is visibly not n^2.

Homework

1. Trace closest pair on 12 given points (put a fixed set in the assignment PDF). State the recurrence and the O(n log n) bound.
2. Written: packing argument, 1 page, with a $\delta \times 2\delta$ picture.
3. Written, 4–6 sentences each: Minkowski sum for collision; visibility graph for a path. No code.

Quiz (10 min)

1. Closest-pair time after the strip argument? (2 pts)
2. Why is the strip 2δ wide? (2 pts)
3. Why is the inner loop O(1) per point? (2 pts)
4. $A \oplus B$ is used for what in robotics? (2 pts)
5. Visibility-graph edge exists when? (2 pts)

Common mistakes

- Sorting inside every recursive call (then it is $O(n \log^2 n)$ or worse if sloppy).
- Building the strip from an unsorted list and then sorting (correct but hide the linear merge).
- Checking all pairs in the strip (correct but not the algorithm).
- Using `< δ` vs `≤ δ` inconsistently; pick `<` and stick to it.
- Implementing a visibility graph for the lab. That is a project, not this week.

Board drawings

1. Split, two recursive pairs, δ, the strip.
2. δ/2 disks packed in the $2\delta \times \delta$ neighborhood.
3. Minkowski: square ⊕ disk = rounded square; polygon ⊕ robot = C-obstacle.
4. Two obstacles, start, goal, a few visibility edges.

Week 13 — From 2D algorithms to graphics systems

Time: 75 min lecture + 60 min live coding
Live coding: ray–triangle picking through a BVH
Lab: project checkpoint (vertical slice)
Homework: project only

Timing

Minutes	Do this
0–10	Quiz Week 12
10–30	Algorithm → engine table
30–50	Robustness (epsilon, adaptive, exact)
50–65	What changes in 3D
65–75	Where libraries hide this

Learning goals

1. Map every core algorithm of the course to a graphics / web use.
2. Explain why `EPS` is a policy, not a proof.
3. Name Shewchuk adaptive predicates and when to use a library.
4. List what becomes harder in 3D (hull, Delaunay, predicates).
5. Pick a triangle in a small mesh with a BVH and a ray.

---

1. The payoff table (20 min)

Walk this slowly. For each row: one sentence from the course, one sentence from IGWT.

Algorithm	Graphics / web use	Course week
`orient` / predicates	back-face sign, clipping side, ear test	1–2
Segment intersection	CAD, map overlay, stroke vs stroke, clipping	2, 6
Point in polygon	click a country, select a mesh island in 2D UI	2
Convex hull	OBB / tight 2D collider, silhouette extremes	4–5
Sweep intersections	boolean outlines, font self-overlap	6
Ear clipping	Canvas/SVG fill, UI blobs, simple roofs	7
DCEL / half-edge	mesh edit, subdivision, walk rings	8
kd-tree	2D range, “who is in this tile?”	9
BVH	ray picking, frustum culling, collision broad phase	9, today
Voronoi	stipple, territories, procedural blocks	10
Delaunay / CDT	terrain, navmesh, remesh	11
Closest pair	snap, weld, “too close to merge”	12
Minkowski	character vs wall in configuration space	12
Visibility graph	2D path prototype; navmesh in production	12

Product configurator (the program’s flagship example):
click → ray → BVH → triangle → barycentric coordinates → which part / material. That is Weeks 2 + 9 + today.

Terrain: sites or height samples → Delaunay → vertex normals → a Three.js `BufferGeometry`. Weeks 11 + Computer Graphics I.

Collision: hull or AABB broad phase, SAT or segment tests narrow phase. Weeks 3–6.

---

2. Robustness (20 min)

The problem, again

Orientation of nearly collinear points is a sign of a tiny determinant. IEEE-754 rounds. A predicate that is wrong **once** can:

- flip the wrong Delaunay edge
- invert a hull turn and lose a vertex
- mis-classify a click as outside

`EPS` does not make the predicate exact. It creates a **thick line** where you return COLLINEAR. That is a policy. It can still be inconsistent: A left of BC, B left of CA, C left of AB.

Three levels (teach the names)

Level	What	When
Epsilon	`abs(cross) < EPS`	UI, games, this course's default
Adaptive exact	Shewchuk: start float, recompute with more precision only if the error bound is bad	meshing, CAD
Exact rational / CGAL	integers or rationals, slow	kernels, research

Shewchuk's paper / notes are the reading. We do not reimplement them. For a thesis-quality mesh tool, **call a library**.

Practical course rules

1. Snap inputs (grid) when the UI allows it.
2. Remove duplicates within EPS before hull / Delaunay.
3. One kernel file; do not copy `orient` into five files with five epsilons.
4. If a test fails only on a 1e-12 case, record it; do not "fix" by growing EPS until all tests pass.

---

3. What changes in 3D (15 min)

2D	3D
`orient` 3 points	`orient` 4 points (signed tetra volume)
Segment–segment	segment–triangle, triangle–triangle
Convex polygon	convex polyhedron
Delaunay triangles	Delaunay tetrahedra (sliver tets are a research problem)
Voronoi cells (polygons)	Voronoi cells (polyhedra)
Ear clipping	no simple analog; use CDT or voxel / tet tools
Point in polygon	point in polyhedron (ray cast + solid winding)

Ray–triangle (Möller–Trumbore, teaching).
Ray `o + t d`, triangle abc. Solve for t, u, v barycentric. Hit if t ≥ 0, u ≥ 0, v ≥ 0, u+v ≤ 1.

This is the narrow phase under the BVH.

Mesh repair.
Imported glTF often has: non-manifold edges, opposite windings, duplicate vertices, self-intersections. A DCEL rebuild + Week 6 intersections is the academic version of "click Repair in Blender."

---

4. Who hides this in production (10 min)

Library	What it hides
Three.js	scene graph, not spatial index (unless you add one)
`three-mesh-bvh`	BVH + raycast
cannon.js / rapier / PhysX	collision, often GJK / SAT / hulls
earcut	2D ear clipping (hole support)
Delaunator	2D Delaunay
CGAL	exact predicates, 3D hulls, tets
Recast / Detour	navmesh

Course rule still stands: the project's **main** algorithm is student code. These libraries may render, load models, or provide a reference oracle.

Live coding (60 min)

A mini scene: 20–80 triangles (a box, a ground, a few props). No Three.js required; a 2D projection of 3D is enough, or a simple canvas raycaster.

1. Build a BVH of triangle AABBs (top-down median split on the longest axis).
2. On click, cast a ray.
3. Draw visited boxes in orange, pruned in gray, the hit triangle in green.
4. Print triangle id and barycentric u, v.

Talk: "this is Week 9's prune test plus Week 2's inside-triangle test, in 3D clothing."

Lab — project checkpoint

Each team (2–3) demos a **vertical slice** (10 minutes including setup):

- The core algorithm runs on a non-trivial input.
- Something moves on screen (not a screenshot-only).
- They can name the predicate that would break their demo.

TA / professor checklist:

Check	Yes / no
Algorithm is theirs, not a library	
Visualizer or scene exists	
One degenerate case discussed	
Repo clones and runs	
They know which Weeks they are using	

No grade shock this week: checkpoint is completion + advice. The rubric lives in Week 15.

Homework

Project only. Before Week 14:

- README with build instructions
- List of degenerate cases you handle or document
- A 6-bullet report outline

Quiz (10 min)

1. Which course algorithm does click-to-pick in a configurator use? (2 pts)
2. Why is EPS not exact? (2 pts)
3. What does Shewchuk's method do that EPS does not? (2 pts)
4. 3D analog of `orient(a,b,c)`? (2 pts)
5. Name one library that hides BVH raycast. (2 pts)

Common mistakes

- Using Three.js `Raycaster` for the live-coding "implementation" without a student BVH. Allowed as oracle, not as the demo of the algorithm.
- Growing EPS until Delaunay "looks fine."
- Starting the project this week from zero. The checkpoint exists to prevent that.

Board drawings

1. The payoff table (keep one column empty and fill with the class).
2. Inconsistent epsilon orientations (three points, three disagreeing signs).
3. Ray vs AABB vs triangle.

4. BVH: miss the parent box, never see the children.

Week 14 — Project studio

Time: 30 min lecture, then studio until the period ends
No new theory
Required this week: running algorithm, one degeneracy, README

Timing

Minutes	Do this
0–15	How to write the report
15–30	Defense-style questions; presentation clock
30–end	Desk review. Professor and TAs circulate.

Do not give a third lecture on Delaunay. If a team is stuck on a predicate, debug the predicate.

Learning goals (for the studio)

1. Freeze the project scope: one core algorithm, one visual story.
2. Write a report that a second examiner can grade without watching the demo twice.
3. Answer "what happens on a T-junction / collinear / duplicate?" without guessing.

---

1. Report structure (15 min)

6–8 pages, including figures. Not a blog post. Not a 30-page thesis.

Section	What we want	Typical length
Problem	The geometric question in one paragraph	½ page
Related course ideas	Which weeks, which algorithms you did **not** use and why	½ page
Algorithm	Invariants, complexity, pseudocode or clear steps	2 pages
Degeneracy	At least one case: handle or document	½–1 page
Implementation	Kernel, visualizer, tests	1 page
Results	Screenshots, a timing table if relevant	1 page
Limitations and future work	Honest	½ page
References	de Berg, Ericson, Shewchuk, docs	½ page

Figures must have captions. "Figure 3. Illegal edge before flip."

Do not paste 200 lines of code. A 15-line kernel is enough. The repo is the code.

Do not invent timings. If you did not measure, say so.

---

2. Questions I will ask next week (15 min)

Give students this list. Next week you will actually ask two of them.

1. What is the **predicate** at the center of your project?
2. What is the complexity, and for which n did you try it?
3. Show me a degenerate input. What does your program do?
4. If I replace your main algorithm with the naive one, what do I lose?
5. What would break if we moved this to 3D?
6. Which result is a **construction**, and how do you know the predicate that supports it is right?
7. Point to the test that would fail if `orient` flipped sign.
8. What did a library do, and what did you write?

Presentation clock (Week 15)

- 12 minutes demo + story
- 5 minutes questions
- Hard stop. Rehearse once this week with a TA timer.

Suggested 12-minute shape:

Min	Content
0–2	Problem and one picture
2–6	Algorithm, one invariant, one complexity sentence
6–10	Live demo, including one ugly input
10–12	Limitations, who did what

---

3. Studio rules

Must be true before they leave today

- `npm start` / `python -m http.server` / whatever is in the README works on a lab machine.
- The core algorithm runs on more than 5 toy points.
- At least one degenerate case is either handled or shown and explained.
- README: install, run, who implemented which file.

Professor / TA desk review (10 minutes per team)

Look at, in this order:

1. The kernel file (`orient`, intersection, incircle, …)
2. The visualizer
3. Tests
4. The report outline

Comment on the **algorithm**, not the CSS.

Scope cuts (offer these; do not let teams “add AI”)

If they are behind	Cut
Fortune half-done	Discrete Voronoi + Delaunay dual
Full map editor	Intersection + DCEL walk, no undo stack
3D physics	2D hull + SAT
Pathfinding + rendering	Visibility graph on a 2D floor plan
Beautiful shaders	One unlit canvas and a correct kernel

A correct O(n log n) algorithm on a plain canvas beats a broken Three.js scene.

Lab

The studio **is** the lab. Attendance is required. Checkpoint grade: complete / incomplete from Week 13, plus today’s README/degeneracy check.

Homework

Finish the report draft and the 30-second screen recording (portfolio). Submit the recording before Week 15 so the session cannot die on a failed HDMI cable.

Quiz

None.

Common failure modes this week

- Pretty UI, no tests.

- Library did the Delaunay; student drew it.
- Report is a tutorial on Three.js.
- “It works on my laptop” and no lockfile / no README.
- Three people, one git author.

Board

Write only:

1. The 8 report headings
2. The 8 defense questions
3. The 12 + 5 clock

Week 15 — Project presentations

Time: full session of 12 + 5 minute slots
No new lecture
Deliverables: live demo, repo, 6–8 page report, 30-second recording

---

Before the session

- Room: projector + one spare HDMI/USB-C dongle.
- Load each team's recording as backup.
- Print the rubric (below) one sheet per team.
- Order: random or by project type (all Voronoi, then all picking) so comparison is fair.
- A TA keeps time: 10-minute warning, 12-minute stop.

Slot format

Time	Who	What
12 min	team	problem, algorithm, live demo, limitations
5 min	examiner	two questions from the Week 14 list, plus one specific to their code
1 min	—	switch

If the demo dies, play the recording and continue. Do not debug for five minutes in front of the class.

Required deliverables

1. **Live demo** of the student algorithm.
2. **Repository** with README, kernel, tests, and a note on libraries.
3. **Report** 6–8 pages, captions on figures.
4. **30-second recording** for the IGWT portfolio / exhibition.

Late report: follow the department rule you announced in Week 1. Suggested: −10% per 24 hours, zero after 72 hours.

Rubric (30% of the course)

Copy onto the grade sheet.

Criterion	Weight	5	3	1
Correct algorithm (tests + degeneracy)	30%	Kernel matches the lecture invariant; tests include a degenerate case	Algorithm works on happy input; weak tests	Library does the work, or wrong algorithm
Visual explanation	20%	Viewer shows the invariant (stack, strip, illegal edge, sweep, …)	Result is visible, process is not	Screenshots of a finished mesh only
Code and repository	15%	Runs from README; one kernel; clear authors	Runs with tribal knowledge	Does not run on the lab machine
Report	20%	Problem, complexity, degeneracy, honest limits, citations	Write-up exists but skips complexity or degeneracy	Tutorial padding, no figures
Presentation and demo	15%	12 minutes, ugly input shown, questions answered	Demo works, story is fuzzy	Over time, or cannot answer the predicate question

Half-points are allowed. Three teammates may receive different participation adjustments if the repo history and the Q&A make that obvious. Announce this in Week 1.

Suggested questions by project type

Map editor. How do you report a T-junction? Do you split the DCEL edge?

Polygon modeler. Ear or CDT? Show a reflex vertex that is not an ear.

Terrain / city. Dual: which Voronoi vertex is which Delaunay triangle? Empty circle?

Physics. Hull policy for collinear? SAT vs segment tests?

Picking / configurator. What do you prune? Barycentric meaning for "which part"?

Path demo. Visibility edge vs Voronoi roadmap. Shortest vs safest?

Stipple. Discrete vs exact Voronoi. What is a vertex in the pixel picture?

Mesh repair. Which intersections do you insert? Is the result a valid DCEL?

Always ask: **"Where is `orient` (or incircle) in your repo?"**

After the session

- Collect repos as tags / zips.
- Pick 3–4 demos for the annual IGWT exhibition.
- Note one curriculum fix for next year (where did everyone break?).
- Typical fixes: more time on `onSegment`, a provided visualizer, a smaller n cap on Delaunay.

What you say in the last two minutes of the course

Computational geometry in this program is not a museum of algorithms.

It is the habit of:

1. naming the predicate,
2. drawing the invariant,
3. testing the degenerate case,
4. only then putting the result on a GPU.

That habit is what they should take into Computer Graphics I, WebGL, and the capstone.

Quiz / homework

None. Grades close when the report and repo are in.

Board

The rubric table. The clock. Nothing else.